

Using AI To Classify Radio Frequency Emanations

UT Dallas Team 2

Adithya Chintala – Student Researcher – Adithya.Chintala@UTDallas.edu

Brigham Thornock – Student Researcher – Brigham.Thornock@UTDallas.edu

Ian Zhang – Student Researcher – ian.zhang@utdallas.edu

Ryan Nelson – Student Researcher – Ryan.Nelson@UTDallas.edu

Samuel Ofiaza – Student Researcher – Samuel.Ofiaza@UTDallas.edu

Brian Ricks – Faculty Support – absolutefunk@utdallas.edu

Problem Mentors:

Brad Kline - National Security Agency - bjkline@uwe.nsa.gov

Drew Wilson - National Security Agency - aewilso@uwe.nsa.gov

Sam Osofsky - National Security Agency - stosofs@radium.ncsc.mil

Investigator Biographies

Adithya Chintala:

MS in CS (Cyber). SFS. INL Intern.

Focus: Anomaly Detection, Adversarial AI.

Ian Zhang:

MS in CS (AI/ML). Toyota & F5 Intern.

Focus: Crypto, AI/ML Applications.

Brigham Thornock:

MS in CS (Cyber). SFS, IDA Intern.

Focus: ML, Offensive Security.

Samuel Ofiaza:

MS in CS (Cyber). SFS, INL Intern.

Focus: Adversarial ML.

Ryan Nelson:

MS in CS (Cyber). SFS, LANL Intern.

Focus: Hardware security, RF, and ML.

Agenda

- Investigator Biographies
- Project Introduction
- Project Objectives
- Research Plan and Progress to Date
- Model Results
- Issues/Challenges + Next Steps

Project Introduction

The Problem:

- Electronic systems emit unintended Radio Frequency (RF) signals.
- Adversaries can intercept these signals to reconstruct sensitive data.
- This attack bypasses traditional encryption and software defenses by targeting the physical layer.

Our Research Question:

Can AI/ML models reliably identify and classify data patterns (e.g. characters) from these RF emanations?

We aim to build an AI pipeline to demonstrate this vulnerability and explore its limits.

Project Objectives

Primary Goal:

Develop a reliable AI pipeline for character classification from RF signals.

Specific Objectives:

- Build a robust data processing pipeline for I/Q samples.
- Develop and optimize multiple ML architectures (CNN, RNN, etc.)
- Test models on progressively difficult datasets.
- Create a final, high-performance ensemble model.

Research Plan and Progress to Date

- **Data:**

- Template pickle files: contains a “snapshot” of the signal of specific characters/bytes (Ex. signal for the character ‘A’)
 - Unpickled and converted to dataframes (Pandas/Polars) for easier workability
- Sequence pickle files: contains a signal of a series of characters/block of text

- **Models Considered:**

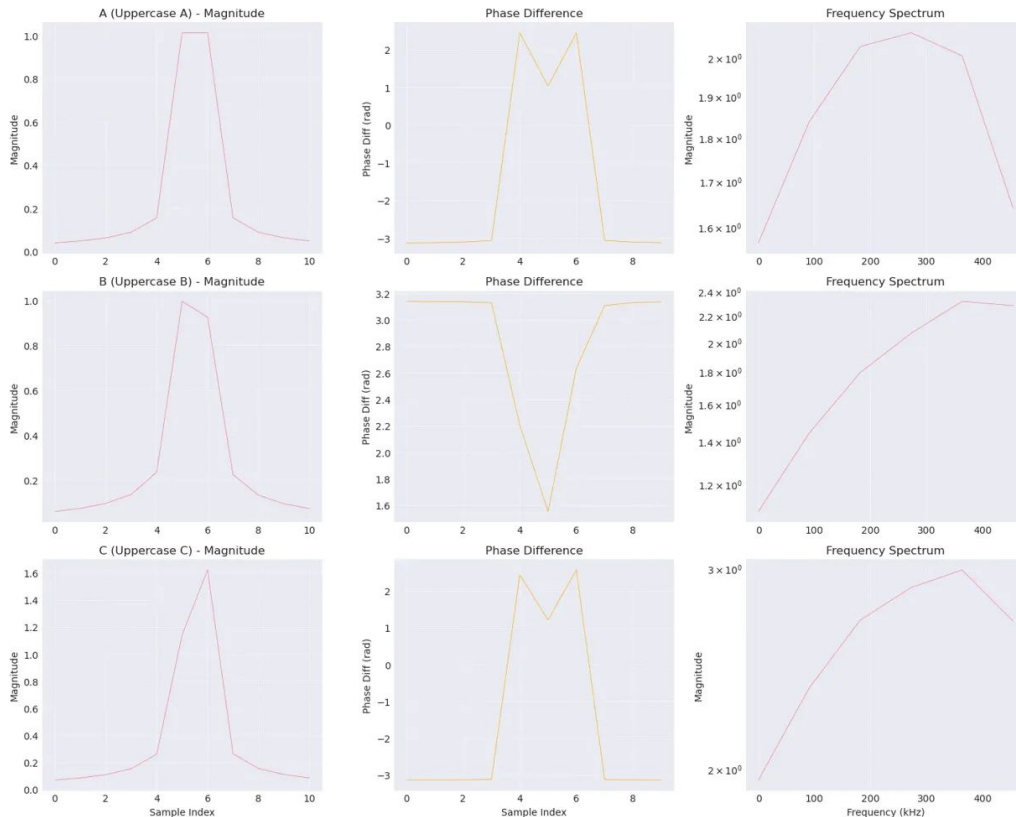
- Deep Learning Models: CNNs, LSTMs, MLPs
- Traditional Supervised Models: SVMs (Support Vector Machines), Random Forest, Naive Bayes, Decision Trees, KNN (K-Nearest Neighbors), GBM (Gradient Boosting Machines)

- **Progress:** Preprocessed the data and tested these models on only the template pickle files (individual characters).

Part 0: Data Visualization

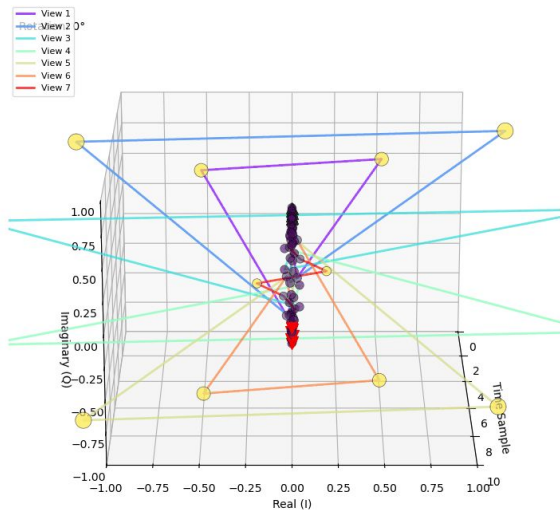
Dataset Overview

- **Data Type:** Complex-valued RF emanations from computers processing text (unintended electromagnetic leakage)
- **Signal Structure:** 11-sample complex signals per character, sampled at 1 MHz
- **Multi-view Recording:** 7 simultaneous RF receivers at different spatial positions capturing the same emanation
- **Dataset Scale:** 128 ASCII characters $\times$ 7 views $\times$ 4 recordings = 3,584 samples per file
- **Noise Conditions:** 5 levels (NL00-NL04) ranging from clean to 30% bit error rate
- **Character Density:** 3 padding levels (CD00-CD02) with 0, 4, or 10 NULL symbols between characters



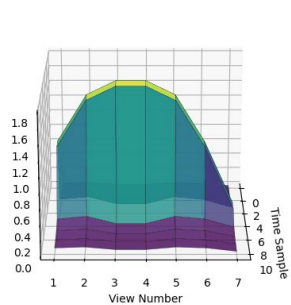
Part 0: Data Visualization

Letter 'A' - 3D RF Emission Signature
7 Receiver Views

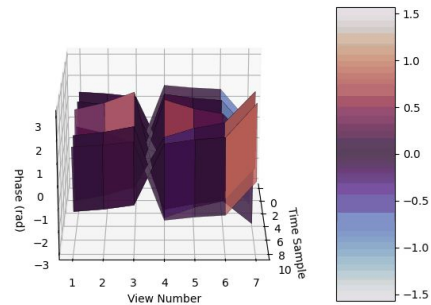


Letter 'A' - Multi-View RF Emission Analysis
Rotation: 0°

Magnitude Surface Across Views

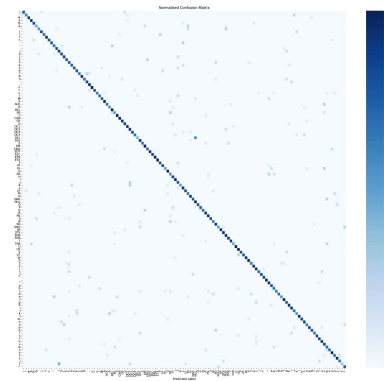
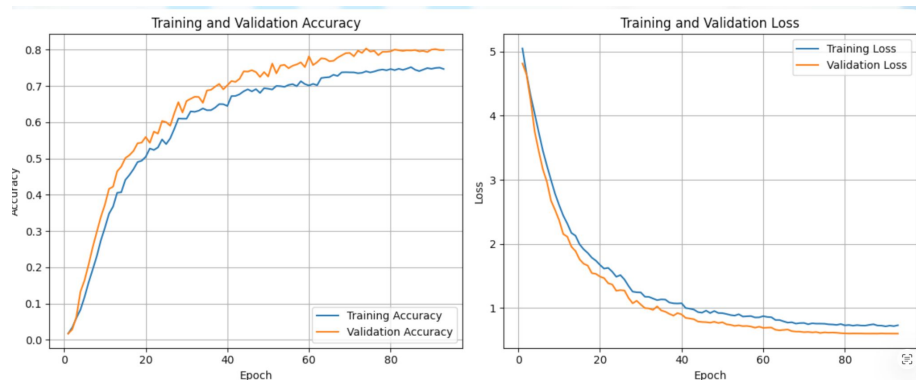


Phase Surface Across Views



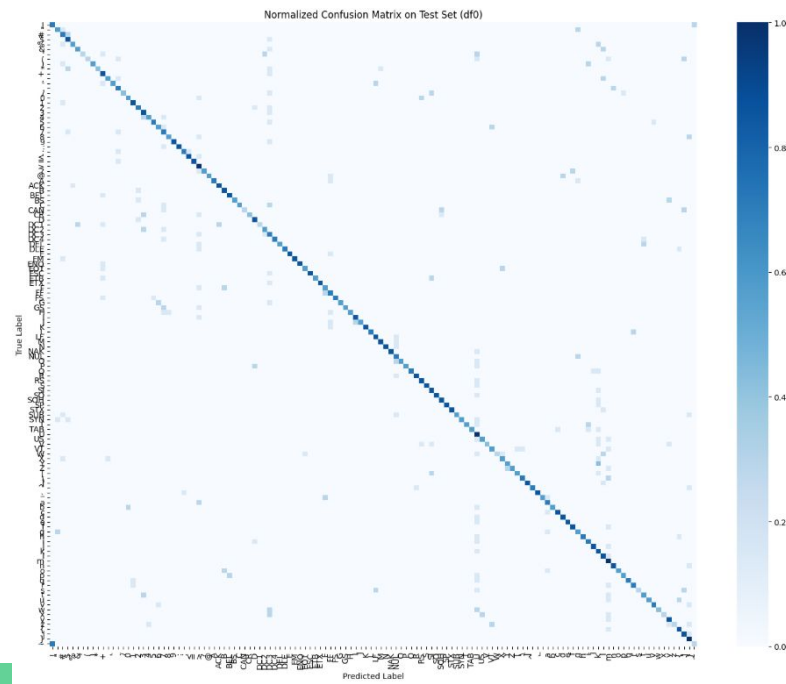
Part 1: CNN Approach

- Ideal candidate for detecting patterns in the context
- Input: 11 (padding + character) * 2 (in-phase and quadrature components)
- Output: Multi-class classifier
- Result: 0.80 accuracy, 0.82 precision, 0.80 recall, and 0.80 F1 score
- Inconsistent result for individual characters: 0.44 - 1.00 accuracy



Part 2: The LSTM + MLP Hybrid Approach

- LSTM (Long Short-Term Memory): good for sequential data
- MLP (Multi-Layer Perceptron): good for static/categorical data
- Hybrid approach: best of both worlds
- Result: ~68% accuracy
- Takeaways:
 - Better hyperparameters
 - Better training/testing methodology
 - Model wasn't the best fit?



Part 3.1: Multilayer Perceptron (MLP)

Layer (type:depth-idx)	Input Shape	Output Shape	Param #
Sequential	[1, 22]	[1, 128]	--
└Linear: 1-1	[1, 22]	[1, 512]	11,776
└ReLU: 1-2	[1, 512]	[1, 512]	--
└Dropout: 1-3	[1, 512]	[1, 512]	--
└Linear: 1-4	[1, 512]	[1, 512]	262,656
└ReLU: 1-5	[1, 512]	[1, 512]	--
└Dropout: 1-6	[1, 512]	[1, 512]	--
└Linear: 1-7	[1, 512]	[1, 256]	131,328
└ReLU: 1-8	[1, 256]	[1, 256]	--
└Dropout: 1-9	[1, 256]	[1, 256]	--
└Linear: 1-10	[1, 256]	[1, 256]	65,792
└ReLU: 1-11	[1, 256]	[1, 256]	--
└Dropout: 1-12	[1, 256]	[1, 256]	--
└Linear: 1-13	[1, 256]	[1, 128]	32,896
Total params: 504,448			
Trainable params: 504,448			
Non-trainable params: 0			
Total mult-adds (Units.MEGABYTES): 0.50			
Input size (MB): 0.00			
Forward/backward pass size (MB): 0.01			
Params size (MB): 2.02			
Estimated Total Size (MB): 2.03			

	precision	recall	f1-score
!	0.928571	1.000000	0.962963
"	0.900000	1.000000	0.947368
#	0.689655	0.833333	0.754717
\$	1.000000	0.535714	0.697674
%	0.619048	1.000000	0.764706
...	...	...	...
}	0.923077	0.705882	0.800000
~	0.720000	0.818182	0.765957
accuracy	0.830233	0.830233	0.830233
macro avg	0.849612	0.839277	0.832103
weighted avg	0.855835	0.830233	0.830592

Part 3.2: Traditional Supervised Learning Models

Supervised Classification Model	Real/Imaginary Raw Padded	Magnitude/Phase Raw Padded
Naive Bayes	0.04	0.02
Decision Tree	0.97	0.94
Random Forest	0.99	0.95
SVM	0.25	0.20
KNN	0.20	0.28
XGBoost	0.96	0.95

Issues/Challenges + Next Steps

Issues/Challenges

- Model performance is based on “clean” data, which is not realistic
 - Might reduce on “harder” data containing noise
- Project sponsors not available due to government shutdown
 - Lack of guidance and clear indication of future steps

Next Steps

- Repeat experiments on the “harder” data
 - Unpickle the sequence files and test existing and new models
 - Optimize hyperparameters
- Create an ensemble of the best working models

Bibliography

Papers:

- [1] Agarwal, A. (2019, June 11). Signal Modeling Using Recurrent Neural Networks. Retrieved September 12, 2025, from Medium website: <https://pub.towardsai.net/signal-modelling-using-recurrent-neural-networks-d832f0c50503>
- [2] Kiranyaz, S. (2021). 1D convolutional neural networks and applications: A survey. *Mechanical Systems and Signal Processing*, 151, 107398. <https://doi.org/10.1016/j.ymssp.2020.107398>
- [3] Shevitski, B., Watkins, Y., Man, N., & Girard, M. (2023). Digital Signal Processing Using Deep Neural Networks. *OSTI OAI (U.S. Department of Energy Office of Scientific and Technical Information)*. <https://doi.org/10.2172/1984848>

Acronyms

AI – Artificial Intelligence

ML – Machine Learning

RF – Radio Frequency

NN – Neural Network

CNN – Convolutional Neural Network

RNN – Recurrent Neural Network

LSTM – Long Short-Term Memory

I/Q – In-phase / Quadrature

DOE – Department of Energy

PNNL – Pacific Northwest National Laboratory

UTD – The University of Texas at Dallas

FGSM – Fast Gradient Sign Method

GAN – Generative Adversarial Network

AWS – Amazon Web Services

PKI – Public Key Infrastructure